MASTER UNIVERSITARIO EN DESARROLLO ÁGIL DE SOFTWARE PARA LA WEB

ASIGNATURA: INTEGRACIÓN CONTINUA EN EL DESARROLLO ÁGIL

CURSO 2025-26

TRABAJO FINAL

AUTOR/A: Ismael Blázquez Carabias (ismael.blazquez@edu.uah.es)

Contenido

Enlaces Relacionados 3
Apartado 1) 4
Apartado 2) 4
Apartado 3) 5
Apartado 4) 8
Apartado 5) 9
Apartado 5.1) 9
Apartado 5.2) 10
Apartado 5.3) 11
Apartado 5.4) 12
Apartado 5.5) 13
Apartado 5.6) 15

Enlaces Relacionados

URL del **repositorio moviecards** en github.com:

https://github.com/blazquez-isma-uah/uah_ic-moviecards

URL de la **aplicación moviecards**:

https://moviecards-ismael.azurewebsites.net/

URL de la **aplicación moviecards** en entorno **pre-producción**:

https://moviecards-ismael-pre.azurewebsites.net/

URL del **repositorio moviecards-service** en github.com:

https://github.com/blazquez-isma-uah/uah_ic-moviecards-service

URL de la **aplicación moviecards-service**:

https://moviecards-service-blazquez.azurewebsites.net

URL de un **vídeo corto** en el que se explique cómo se ha desarrollado el trabajo:

BlazquezCarabiasIsmael_TrabajoFinalICDA2025.mp4

Apartado 1)

Para este apartado se ha seguido la práctica guiada del tema 5. Aplicación en: https://moviecards-ismael.azurewebsites.net/

Apartado 2)

Para este apartado se sigue el mismo procedimiento de la práctica guiada:

Se descarga el proyecto de GitHub de https://github.com/josehilera/moviecards-service al PC local y se crea repositorio local y en GitHub: https://github.com/blazquez-isma-uah/uah_ic-moviecards-service

Se hace un primer commit de toda la aplicación excepto el fichero de workflow main.yaml para que en el repositorio de GitHub se cree el proyecto y la rama main.

Se accede al portal de Azure y se crea una nueva aplicación web y se configura de la misma manera que en la práctica guiada: Marcar “Credenciales de publicación de autenticación básica de SCM” y configuración de la implementación para GitHub y el proyecto nuevo creado.

Como ya existe el main.yaml en el proyecto con el flujo de trabajo completo para la integración continua, se mantienen todos los trabajos excepto el “deploy” que se actualiza con la información generada por azure: “app-name” y “publish-profile” del paso “Deploy to Azure Web App”.

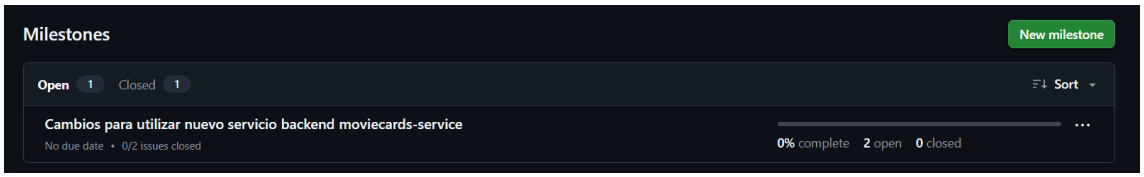
Se tienen que actualizar las versiones de algunos steps ya que las existentes están deprecadas. Y se añade condición para que el trabajo de Deploy solo se realice cuando se hagan cambios sobre la rama main.

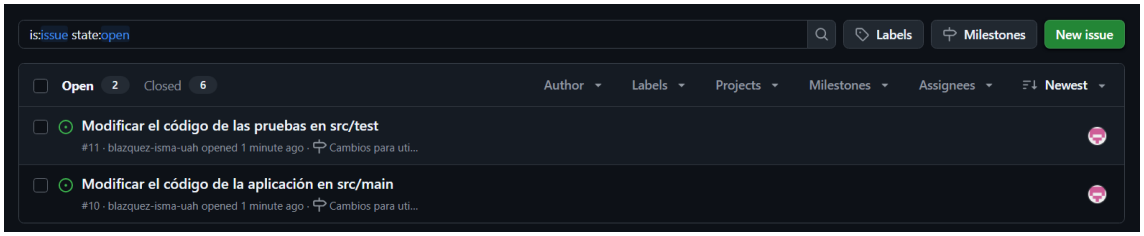
Se deja la aplicación con “despliegue continuo” sin necesidad de aprobación manual.

Se realizan pruebas de todas las operaciones mediante la herramienta Postman.

Apartado 3)

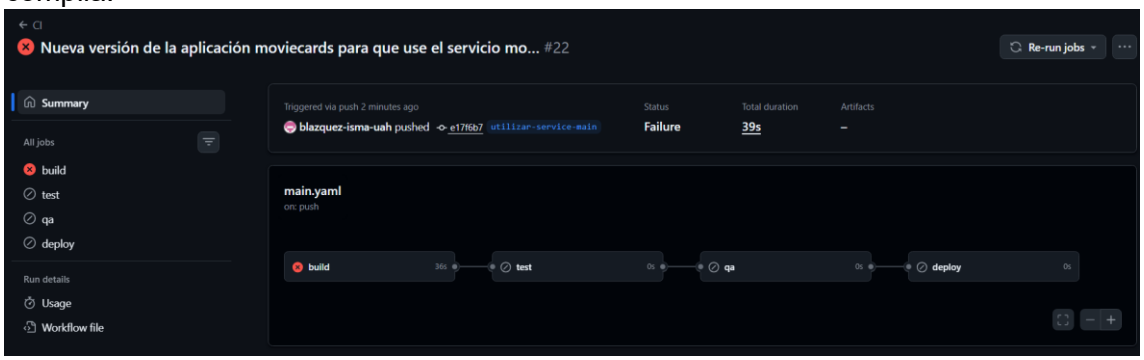
Primero se crea un nuevo Milestone al que se ha llamado “Cambios para utilizar nuevo servicio backend moviecards-service” con las 2 Issues requeridas:





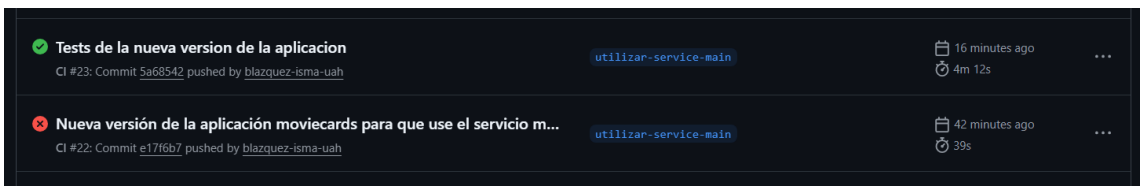
Se pone “In Progress” la Issue para modificar la aplicación, se crea una nueva rama “**utilizar-service-main**” y se hacen los cambios de código indicados en ella.

Al subir los cambios en la rama, se puede ver como el workflow falla en el primer paso “build” porque los tests se corresponden con el código anterior y el proyecto no compila.



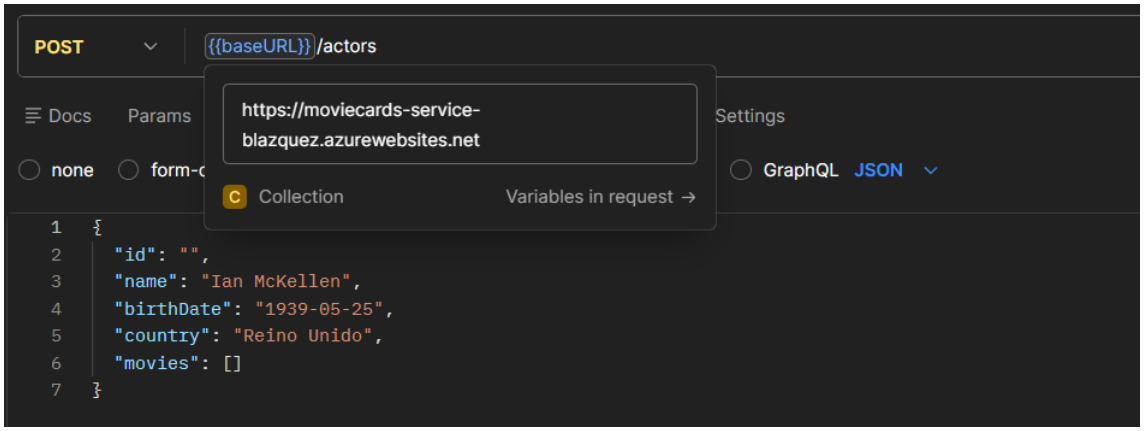
Avanzamos la Issue para modificar los tests al estado “In Progress” y, sobre la misma rama “**utilizar-service-main**”, se hacen los cambios de código indicados en los tests.

Una vez realizados los cambios en los tests, vemos como el workflow deja de fallar.



Cuando se completa el workflow, se hace Pull Request a la rama principal y se completa el deploy. También se cambia el estado de las dos Issues a “Done” y se cierra el Milestone.

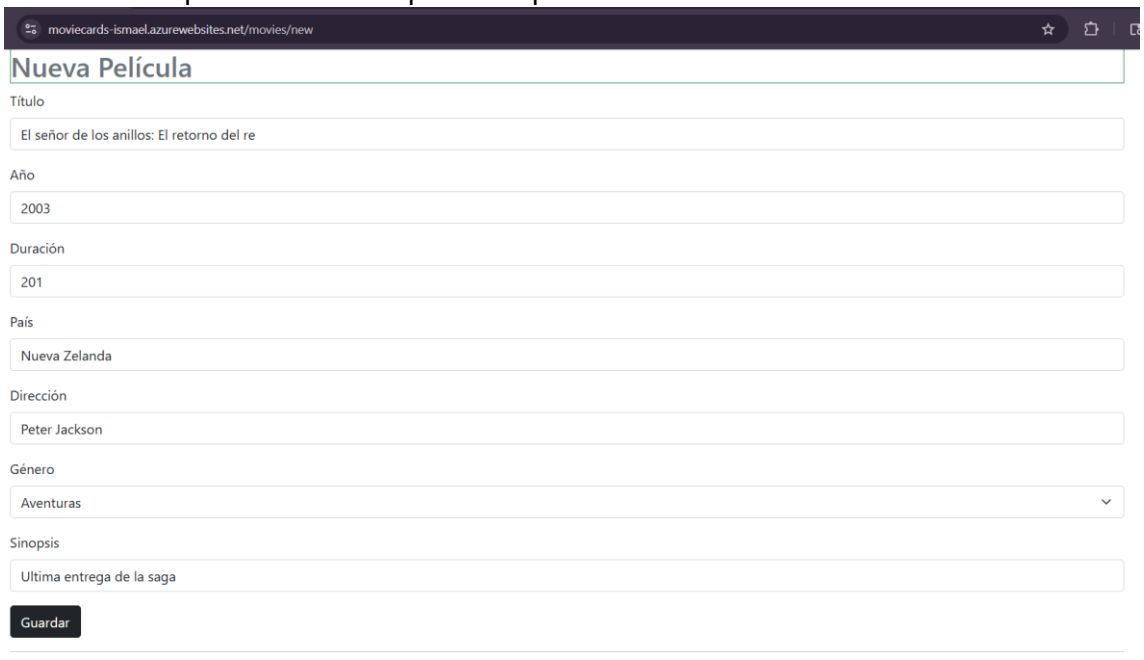
Pruebas del correcto funcionamiento:
Se crea un actor desde Postman directamente en moviecards-service:



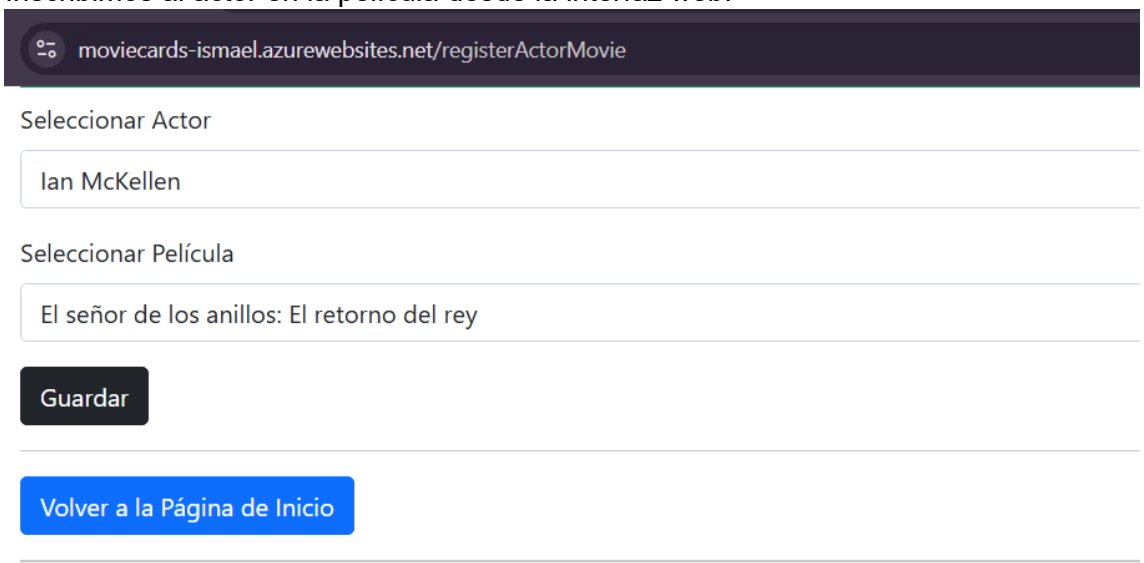
Y vemos como aparece en moviecards



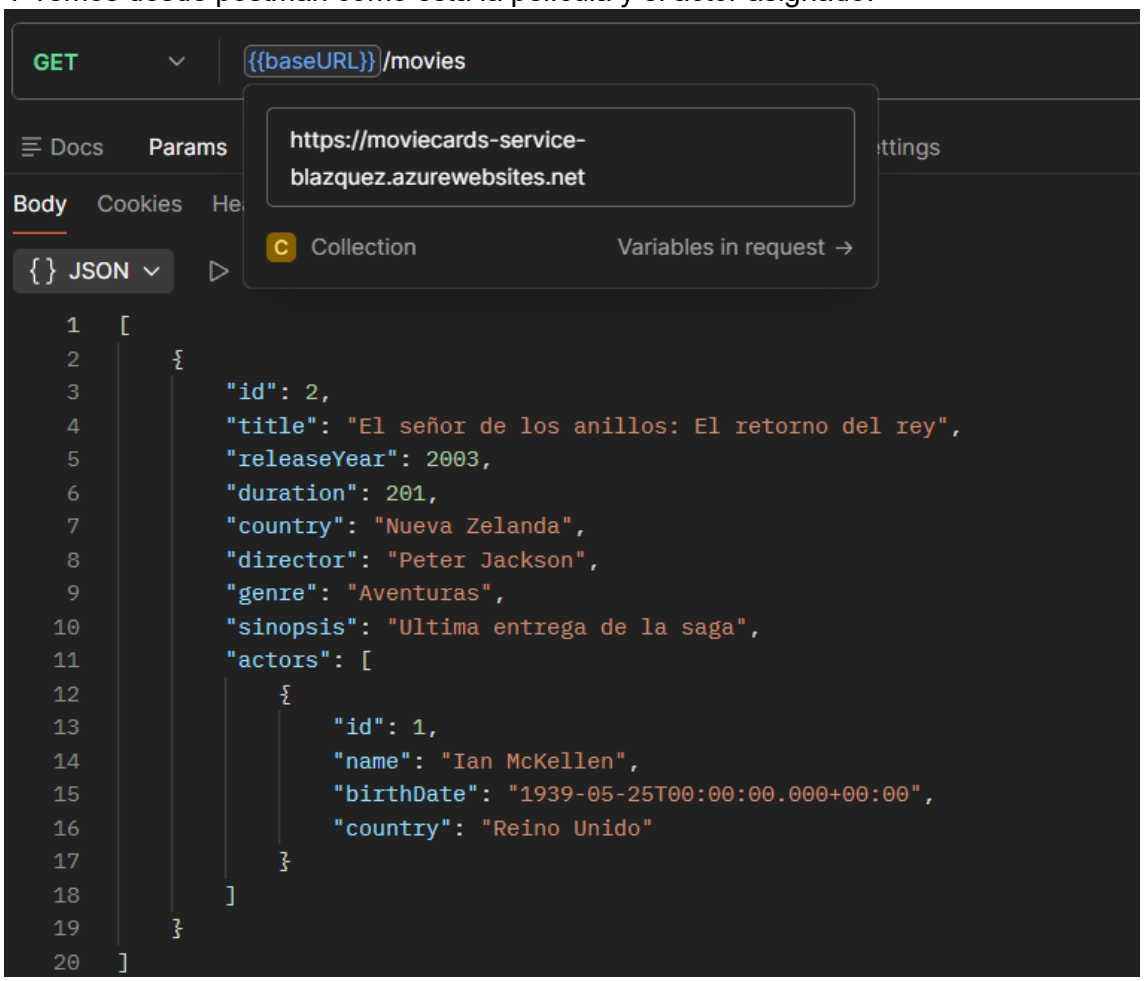
Hacemos la operación inversa para una película. La creamos desde la interfaz web:



Inscribimos al actor en la película desde la interfaz web:



Y vemos desde postman como está la película y el actor asignado:



Apartado 4)

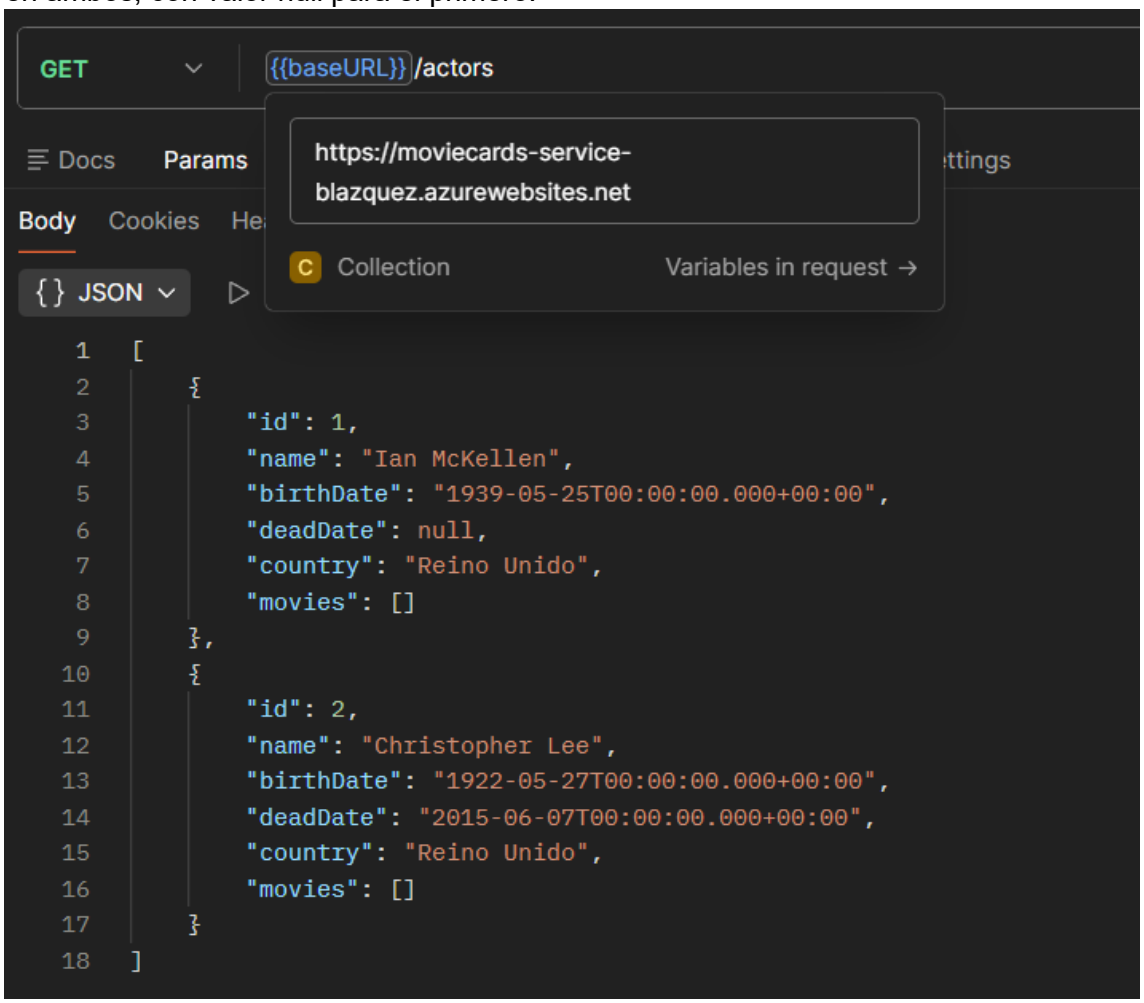
Para añadir el nuevo atributo deadDate se modifican las siguientes clases:

Actor.java: Se añade el nuevo atributo deadDate igual que el atributo birthDate, así como sus métodos get y set y se añade el nuevo atributo al método equals y al método hasCode.

ActorTest.java: Se añade nuevo método test testSetGetDeadDate para testear los metodos get y set.

ActorJPAIT.java: Se añade nuevo atributo deadDate a los test ya existentes testSaveActor y testFindById.

Para probar el correcto funcionamiento, creamos un nuevo actor sin incluir el nuevo atributo deadDate y otro incluyéndolo, y vemos como para el nuevo atributo aparece en ambos, con valor null para el primero:



Apartado 5)

Se crea un nuevo Milestone y 5 Issues asociadas al mismo



Apartado 5.1)

Se pone la Issue "Añadir nuevo trabajo "stage" al workflow" en estado "In Progress".

Para este apartado, lo primero se accede al portal de Azure, se crea una nueva aplicación web con nombre "**moviecards-ismael-pre**" y se configura de la misma manera que en la práctica guiada: Marcar "Credenciales de publicación de autenticación básica de SCM" y configuración de la implementación para GitHub y el proyecto nuevo creado.

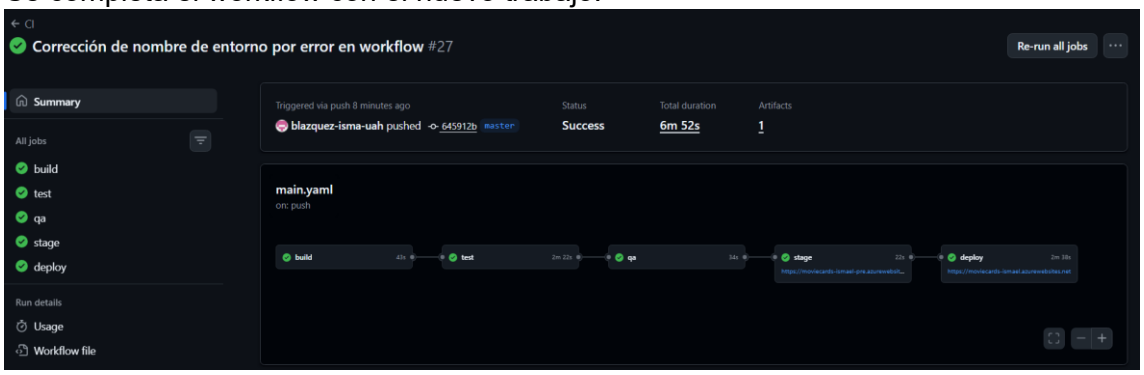
Una vez creado se añade el contenido generado por Azure a un nuevo trabajo "stage" que dependa de "qa" y se actualiza el trabajo "deploy" para que dependa de "stage":

```yaml
  stage:
    runs-on: ubuntu-latest
    needs: qa # Asegura que el despliegue solo ocurra si el trabajo de QA se completa
    environment:
      name: 'Production'
      url: ${{ steps.deploy-to-webapp.outputs.webapp-url }}
    steps:
    - name: Download artifact from build job
      uses: actions/download-artifact@v4
      with:
        name: moviecards-java # Para que coincida con el nombre usado en el trabajo de build

    - name: Deploy to Azure Web App
      id: deploy-to-webapp
      uses: azure/webapps-deploy@v3
      with:
        app-name: 'moviecards-ismael-pre' # https://moviecards-ismael-pre.azurewebsites.net/
        slot-name: 'Production'
        package: '*.jar'
        publish-profile: ${{ secrets.AZUREAPPSERVICE_PUBLISHPROFILE_9826CE1CBE0741AF9C985D67A8C1DFA8 }}

  deploy:
    runs-on: ubuntu-latest
    needs: stage # Asegura que el despliegue solo ocurra si el trabajo de Stage se completa
```

Se completa el workflow con el nuevo trabajo:



Y está disponible la nueva URL https://moviecards-ismael-pre.azurewebsites.net/



Se marca la Issue relacionado como “Done”.

Apartado 5.2)

Se pone la Issue en estado “In Progress” y se crea una nueva “**añadir-fecha-fallecimiento**” rama para el desarrollo.

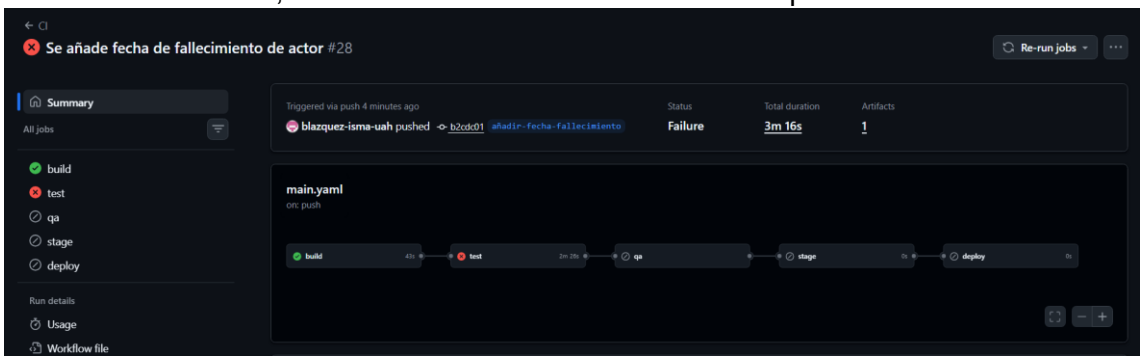
Para añadir el nuevo atributo deadDate se modifican las siguientes clases:

model/Actor.java: Se añade el nuevo atributo deadDate igual que el atributo birthDate, así como sus métodos get y set y se añade el nuevo atributo al método equals y al método hasCode.

actors/form.html: Se añade un input para la fecha de fallecimiento

actors/list.html: Se añade una nueva columna para mostrar el campo actor.deadDate o “-“ si no tuviera

Se hace commit y push a la nueva rama y vemos como el trabajo “test” falla debido a los fallos en los tests, donde no se ha añadido el nuevo campo deadDate.



Se da por completada la Issue pasandola a estado “Done”

Apartado 5.3)

Se pone la Issue en estado “In Progress”. El desarrollo se va a realizar sobre la rama ya creada anteriormente: “**añadir-fecha-fallecimiento**”.

Para añadir el nuevo atributo deadDate a los tests unitarios se modifican las siguientes clases:

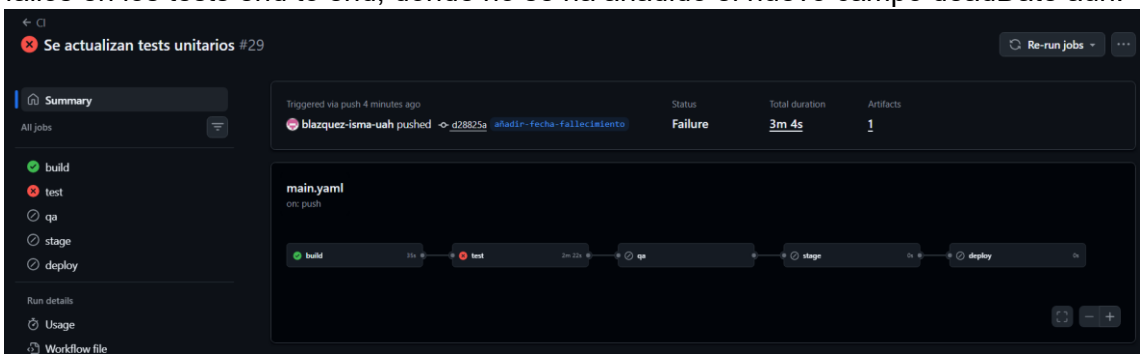
unittest/model/ActorTest.java: Se crea un nuevo método “testSetGetDeadDate” para probar el nuevo atributo creado en el modelo.

Se lanzan todos los tests unitarios para comprobar que están funcionando correctamente:

mvn test -Dtest="com.lauracercas.moviecards.unittest.**"

```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 41, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

Se hace commit y push a la nueva rama y vemos como el trabajo “test” falla debido a fallos en los tests end to end, donde no se ha añadido el nuevo campo deadDate aún.



Se da por completada la Issue pasandola a estado “Done”

Apartado 5.4)

Se pone la Issue en estado “In Progress”. El desarrollo se va a realizar sobre la rama ya creada anteriormente: “**añadir-fecha-fallecimiento**”.

Debido a los cambios realizados en la versión anterior del servicio para que utilizara el nuevo servicio moviecards-service como “fuente de datos”, dejando de utilizar los repositorios propios, hay varias parte del código que ya no tienen utilidad o hay que actualizar.

Para actualizar los tests de integración a la nueva situación se han modificado/eliminado/creado las siguientes clases:

repositories/ActorJPA.java y MovieJPA.java: Ambas clases sobran, ya no se utilizan.

service/actor/ActorServiceImpl.java: Se elimina el import de ActorJPA no utilizado.

service/movie/MovieServiceImpl.java: Se elimina el import de MoveJPA no utilizado.

integrationtest/repositories/ActorJPAIT y MovieJPAIT: Ambas clases sobran ya que ya no hay repositorios JPA.

integrationtest/ActorServiceIT y MovieServiceIT:

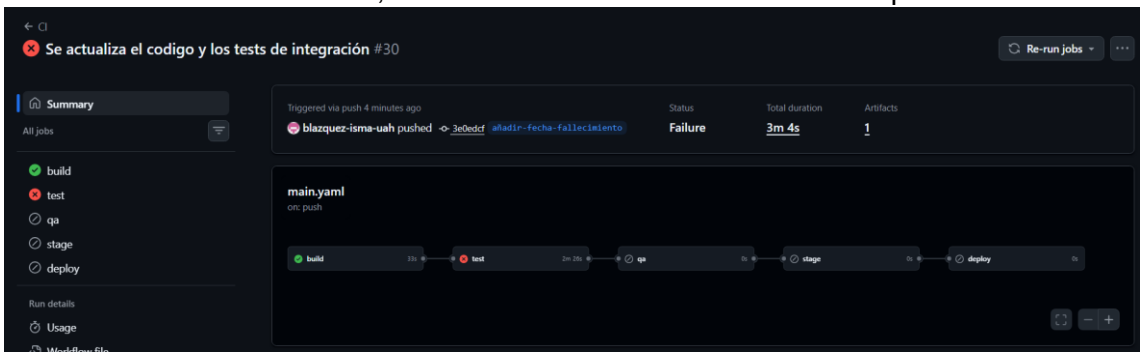
- Nuevas clases generadas para las pruebas de integración de los servicios de Actores y Peliculas
- @SpringBootTest: Carga el contexto completo de Spring Boot, inicializando el bean RestTemplate.
- @MockBean del RestTemplate reemplaza el bean real por un mock para poder simular las respuestas del API del servicio externo sin realizar las llamadas reales.
- Se utiliza Mockito para simular el comportamiento que se espera del RestTemplate.
- Se crean tests para: obtener todos los actores/películas, obtener actores/películas por id, crear y actualizar actores/películas, y manejo de errores 404 y 500.

Con este enfoque se prueba la integración real del servicio dentro del contexto de Spring mockeando solamente la dependencia externa del RestTemplate.

Se lanzan todos los tests de integración para comprobar que están funcionando correctamente:
mvn test -Dtest="com.lauracercas.moviecards.integrationtest.**"

```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  15.128 s
```

Se hace commit y push a la nueva rama y vemos como el trabajo “test” falla debido a fallos en los tests end to end, donde no se ha añadido el nuevo campo deadDate aún.



Se da por completada la Issue pasandola a estado “Done”

Apartado 5.5)

Se pone la Issue en estado “In Progress”. El desarrollo se va a realizar sobre la rama ya creada anteriormente: “**añadir-fecha-fallecimiento**”.

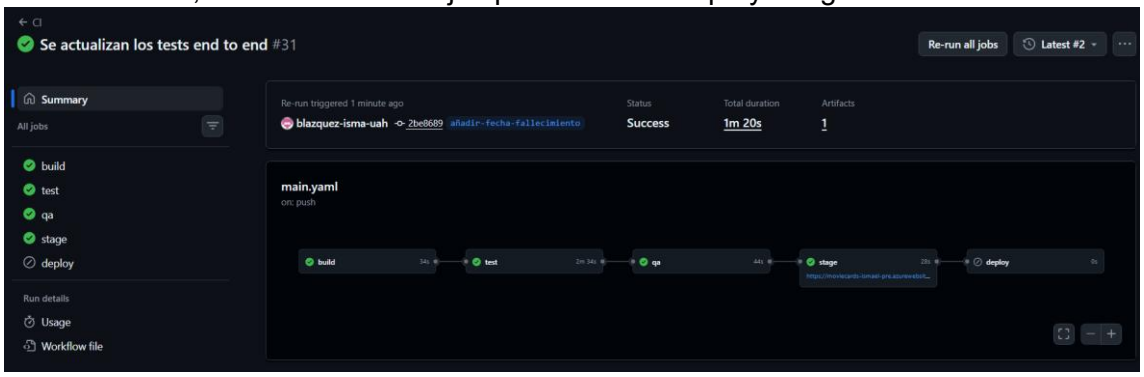
Para añadir el nuevo atributo deadDate a los tests end to end se modifican las siguientes clases:

endtoendtest/ActorE2ETest.java: Se modifican los tests testPageLoad() y testListActors() para que tengan en cuenta el nuevo campo deadDate

Se lanzan todos los tests unitarios para comprobar que están funcionando correctamente:
mvn test -Dtest="com.lauracercas.moviecards.endtoendtest.**"

```
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.468 s - in com.lauracercas.moviecards.endtoendtest
.MovieE2ETest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  16.421 s
```

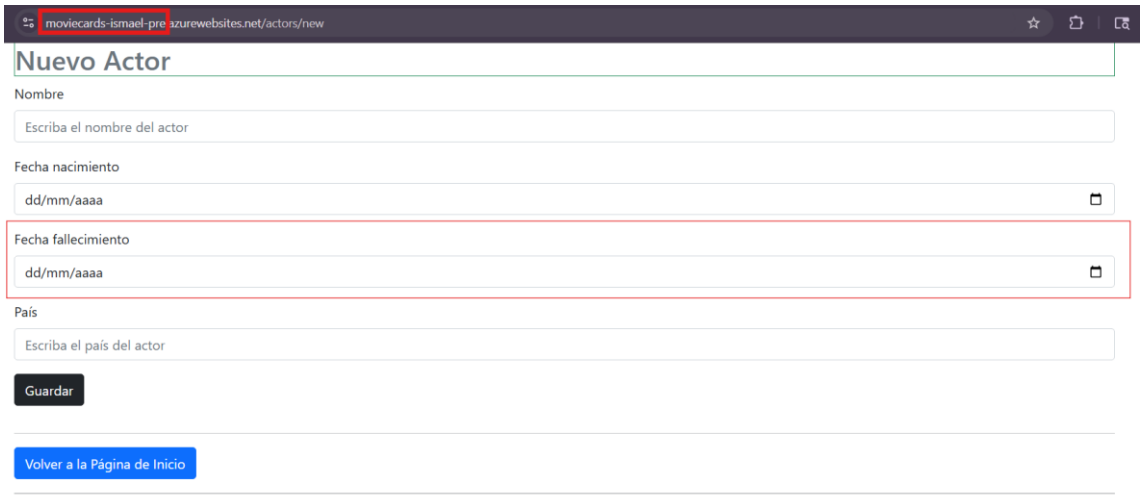
Se hace commit y push a la nueva rama y vemos como el trabajo “test” ya funciona correctamente, así como los trabajos posteriores de “qa” y “stage”.



Se da por completada la Issue pasandola a estado “Done”.

Ahora se puede probar el correcto funcionamiento en el entorno de preproducción que despliega el trabajo “stage”.

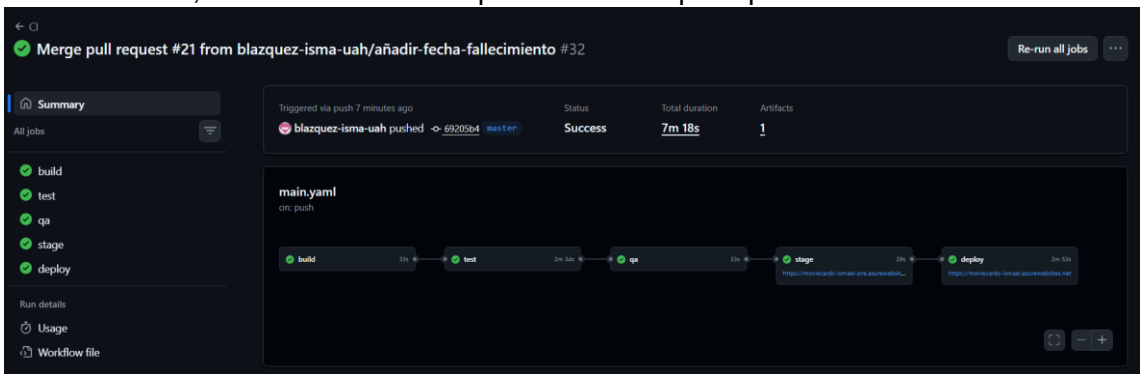
Crear nuevo actor



Listado de actores

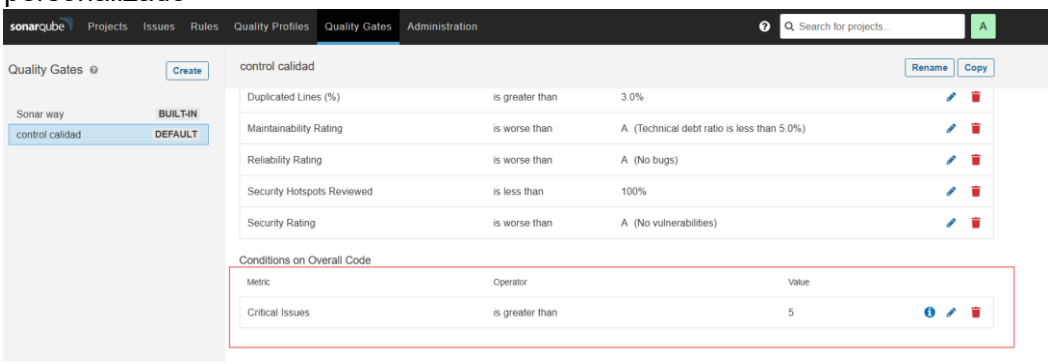


Una vez comprobado que la nueva versión de la aplicación despliega y funciona correctamente, se realiza la Pull Request a la rama principal.



Apartado 5.6)

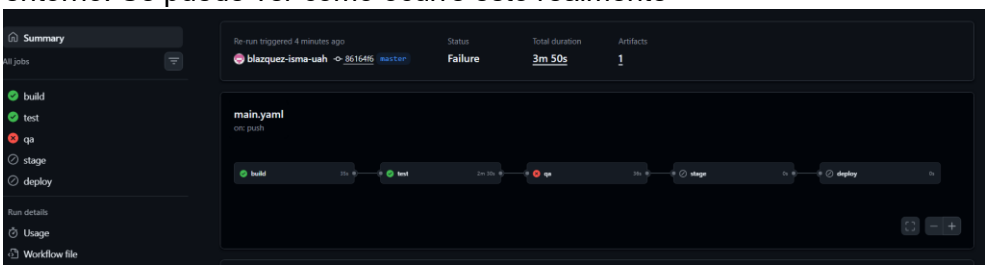
Para detectar y controlar los problemas críticos se crea un criterio en un Quality Gate personalizado



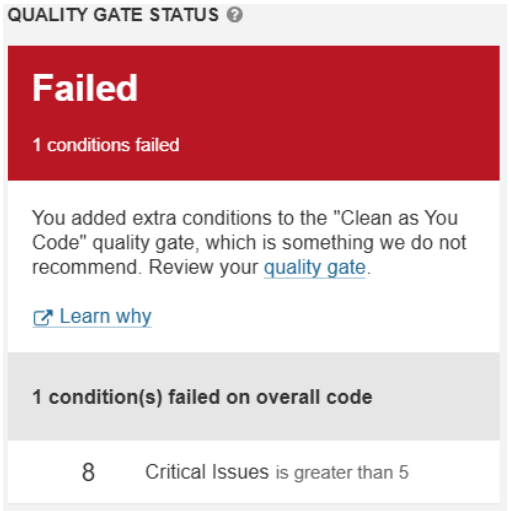
Para controlar que no se despliegue la aplicación en producción si SonarQube falla, se puede eliminar la condición de “continue-on-error: true” del trabajo de “qa” para que falle si se produce un error.

El trabajo “stage” dependerá del correcto funcionamiento de “qa”. Y el trabajo “deploy” del correcto funcionamiento de “stage”.

De esta manera, si “qa” falla, el workflow no continuará y no se desplegará en ningún entorno. Se puede ver como ocurre esto realmente

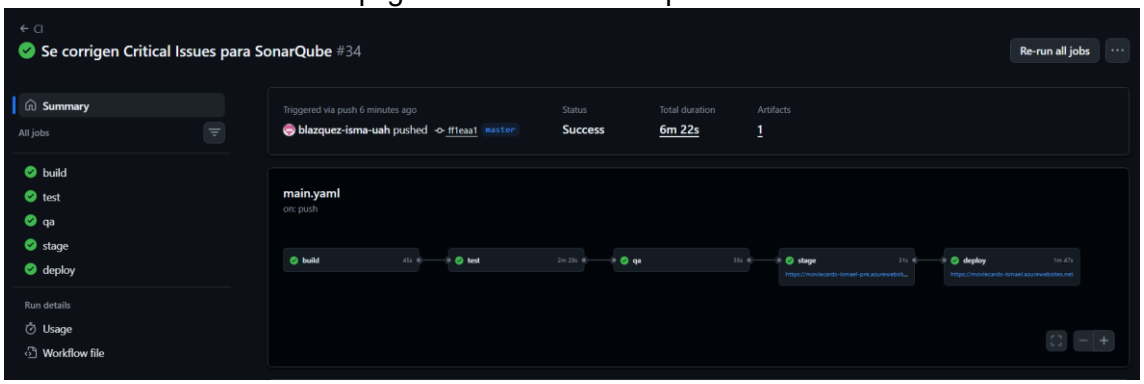


Ya que tiene 8 problemas críticos



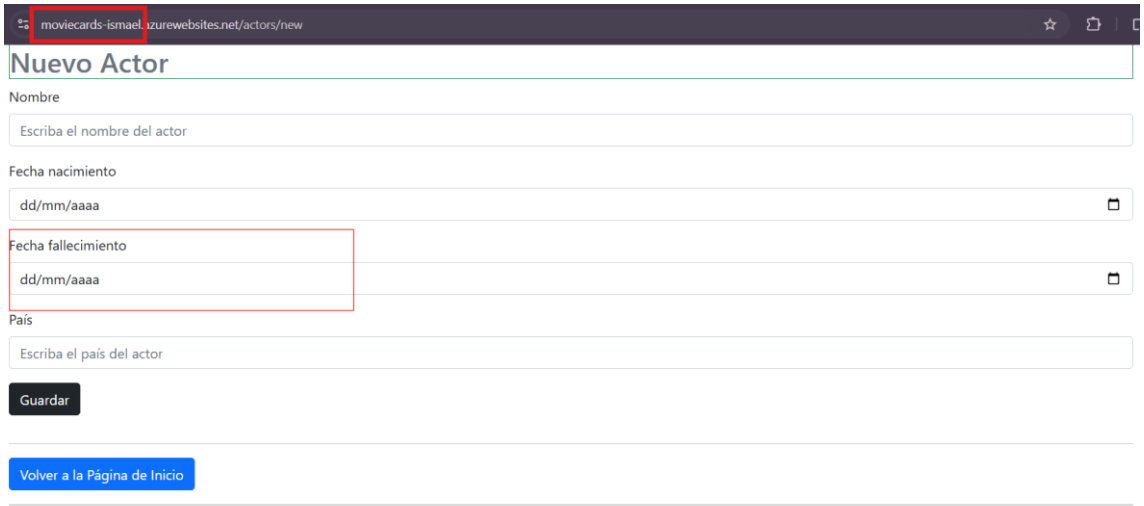
Corregimos varios de los problemas críticos extrayendo a constantes los literales que se repiten en ActorController y MovieController. Una vez hechos los cambios, se suben directamente a la rama master.

Una vez corregidos los problemas críticos, los trabajos del workflow avanzan correctamente hasta el despligue en el entorno de producción.



Se comprueba que en el entorno de producción funcionan los cambios correctamente.

Crear nuevo actor



Listado de actores

